



DOSSIER TECHNIQUE : ECORIDE

Projet Examen ECF - Version 1.0

Novembre - Décembre 2026



Sommaire

01 	CHAPITRE 1 : RÉFLEXIONS INITIALES TECHNOLOGIQUES	Page XX
	1.1 Le Backend : PHP Natif (PHP 8.x)	
	1.2 L'Accès aux Données : PDO (PHP Data Objects)	
	1.3 Le Stockage Relationnel : MySQL	
	1.4 Le Stockage Non-Relationnel : MongoDB	
02 	CHAPITRE 2 : L'ENVIRONNEMENT DE TRAVAIL (MAMP)	Page XX
	2.1 Justification de la pile MAMP	
	2.2 Processus d'Installation et de Configuration	
03 	CHAPITRE 3 : MODÉLISATION DE LA BASE DE DONNÉES (MCD)	Page XX
	3.1 Schéma Graphique du Modèle Conceptuel de Données (MCD)	
	3.2 Dictionnaire des Données Adapté	
	3.3 Règles de Gestion et Cardinalités (MCD)	
04 	CHAPITRE 4 : DIAGRAMME D'UTILISATION (USE CASE)	Page XX
	4.1 Définition des Acteurs	
	4.2 Description Textuelle des Cas d'Utilisation	
05 	CHAPITRE 5 : DIAGRAMME DE SÉQUENCE TECHNIQUE	Page XX
	5.1 Scénario choisi : Réservation d'un trajet par un Passager	
	5.2 Description Textuelle des Échanges (Chronologie)	
	5.3 Scénario alternatif : Consultation et gestion des incidents par un Employé (MongoDB - US 12)	
	5.4 Description Textuelle des Échanges (Chronologie du traitement NoSQL)	
06 	CHAPITRE 6 : DOCUMENTATION DU DÉPLOIEMENT DE L'APPLICATION	Page XX
	6.1 Choix de l'Hébergement et Prérequis	
	6.2 Étapes de Déploiement	

CHAPITRE 1 : RÉFLEXIONS INITIALES TECHNOLOGIQUES

L'architecture technique du projet EcoRide a été pensée pour répondre à des impératifs de performance, de modularité, de sécurité et de robustesse, tout en garantissant une maîtrise totale du code source. Ce chapitre détaille et justifie les choix technologiques retenus pour le backend, l'accès aux données et le stockage.

1.1 Le Backend : PHP Natif (PHP 8.x)

Le choix de PHP natif plutôt que d'un framework lourd (comme Symfony ou Laravel) repose sur plusieurs axes stratégiques :

- **Maîtrise fondamentale des concepts** : le développement en PHP natif permet de manipuler directement la logique de programmation, la gestion des requêtes HTTP, le routage et les mécanismes de session (`$_SESSION`). Cela garantit une compréhension profonde du cycle de vie d'une application web sans l'abstraction imposée par un outil tiers.
- **Légèreté et performance** : sans la surcharge (overhead) d'un framework et de ses nombreuses dépendances, l'application bénéficie de temps d'exécution optimisés et d'une empreinte mémoire réduite, ce qui est idéal pour un déploiement initial fluide.
- **Flexibilité architecturale** : PHP natif offre la liberté de structurer le code à l'état brut (en écrivant les fonctions entières avant d'utiliser des raccourcis), ce qui permet de bien assimiler la séparation des responsabilités entre l'affichage HTML et la logique d'arrière-plan.

1.2 L'Accès aux Données : PDO (PHP Data Objects)

Pour interagir avec le système de gestion de bases de données, l'extension PDO a été préférée aux fonctions MySQLi classiques :

- **Abstraction de la base de données** : PDO agit comme une couche d'abstraction. Si le projet doit migrer de MySQL à un autre SGBD relationnel, le code des requêtes reste globalement identique ; seule la chaîne de connexion (DSN) change.
- **Sécurité native (Requêtes préparées)** : la sécurité est un enjeu majeur pour EcoRide (gestion des comptes, des trajets et des réservations). L'utilisation systématique des requêtes préparées avec PDO (`$pdo->prepare` et `$stmt->execute`) neutralise nativement les risques d'injections SQL en séparant strictement l'instruction SQL des données dynamiques transmises par l'utilisateur.
- **Typage et manipulation des données** : PDO permet de récupérer les données sous forme de tableaux associatifs ou d'objets de manière très structurée, facilitant leur traitement direct en PHP.

1.3 Le Stockage Relationnel : MySQL

Le cœur transactionnel d'EcoRide repose sur MySQL pour les raisons suivantes :

- **Intégrité référentielle et ACID** : Les fonctionnalités de covoiturage exigent une cohérence parfaite des données (un trajet est obligatoirement lié à un conducteur, une réservation est liée à un passager et à un trajet). MySQL garantit le respect des propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité) et gère efficacement les clés étrangères.
- **Robustesse des requêtes complexes** : La recherche de trajets selon des critères multiples (ville de départ, ville d'arrivée, date, options de confort) se prête parfaitement aux capacités d'indexation et aux jointures optimisées de MySQL.

1.4 Le Stockage Non-Relationnel : MongoDB

En complément de MySQL, MongoDB (NoSQL orienté documents) est introduit pour répondre à des besoins spécifiques et distincts du projet :

- **Flexibilité du schéma (Logs et Historiques)** : MongoDB est idéal pour stocker des données dont la structure peut évoluer ou n'a pas besoin de relations strictes, comme les historiques de recherche des utilisateurs ou les logs techniques de l'application.
- **Haute performance en écriture** : pour la journalisation des événements à forte fréquence ou le stockage temporaire, un stockage NoSQL offre des performances en écriture supérieures à une base relationnelle classique, évitant ainsi de surcharger la base de données principale.

CHAPITRE 2 : CONFIGURATION DE L'ENVIRONNEMENT DE TRAVAIL (MAMP)

Pour garantir un développement local homogène, stable et isolé, l'environnement MAMP (Macintosh, Apache, MySQL, PHP) a été sélectionné et configuré.

2.1 Justification de la pile MAMP

MAMP fournit un environnement de développement local complet, évitant d'altérer les configurations système de la machine hôte. Il intègre le serveur web Apache, le système MySQL, ainsi que l'interpréteur PHP, permettant de simuler fidèlement les conditions d'un serveur de production.

2.2 Processus d'Installation et de Configuration

La mise en place de l'environnement s'articule autour des étapes clés suivantes :

A. Téléchargement et Initialisation

- Téléchargement de la version stable de MAMP adaptée au système d'exploitation de la machine de développement.

- Installation dans le répertoire par défaut du système pour éviter les conflits de droits d'accès.

B. Configuration du Serveur Web Apache

Le répertoire racine (Document Root) d'Apache est configuré pour pointer directement sur le dossier de travail du projet EcoRide.

- Cette manipulation permet d'exécuter et de tester l'application en local via une URL propre (ex: `http://localhost:8888/`).

C. Configuration de PHP et de la Base de Données

- **Version de PHP** : sélection de la version PHP 8.x dans les préférences de MAMP pour utiliser les fonctionnalités modernes du langage.
- **Activation des extensions** : vérification dans le fichier de configuration php.ini que les extensions nécessaires (notamment pdo_mysql) sont bien activées pour permettre la communication avec la base de données.
- **Administration** : utilisation de l'outil phpMyAdmin inclus dans MAMP pour créer la base de données initiale, structurer les premières tables et tester les identifiants de connexion (root / root par défaut sur MAMP).

CHAPITRE 3 : MODÉLISATION DE LA BASE DE DONNÉES (MCD)

La modélisation de la base de données d'EcoRide repose sur une approche relationnelle stricte pour le cœur de l'application (MySQL), garantissant qu'aucune réservation ou covoiturage ne puisse exister sans être rattaché à un utilisateur valide et à un véhicule conforme.

3.1 Schéma Graphique du Modèle Conceptuel de Données (MCD)

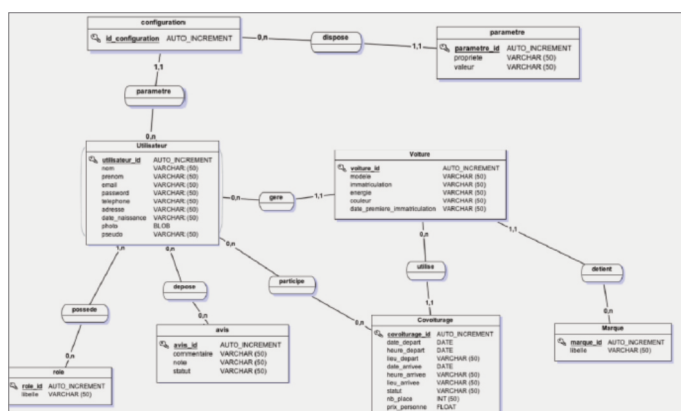


Figure 1 : Représentation graphique du Modèle Conceptuel de Données de l'application EcoRide

3.2 Dictionnaire des Données Adapté

Pour clarifier le rôle de chaque information visible sur le schéma, voici la liste des propriétés et entités qui structurent notre application :

- **Utilisateur :**

- `utilisateur_id` : Identifiant unique automatique (Clé primaire)
- `nom`, `prenom`, `email`, `password`, `telephone`, `adresse`, `date_naissance`, `pseudo` : Informations de profil, de contact et de connexion.
- `photo` : Chaîne de caractères (VARCHAR) contenant le chemin relatif / l'URL de l'image de l'avatar stockée sur le serveur.

- **Voiture :**

- `voiture_id` : Identifiant unique (Clé primaire)
- `modele`, `immatriculation`, `energie`, `couleur`, `date_premiere_immatriculation` : Caractéristiques du véhicule géré par le conducteur.

- **Covoiturage (Trajet) :**

- `covoiturage_id` : Identifiant unique (Clé primaire)
- `lieu_depart`, `lieu_arrivee` : Localisations géographiques du trajet.
- `date_depart`, `heure_depart`, `date_arrivee`, `heure_arrivee` : Informations temporelles précises.
- `nb_place` : Nombre de sièges disponibles à la vente.
- `prix_personne` : Tarif unitaire par passager (nombre à virgule/FLOAT).
- `statut` : État d'avancement du trajet.

- **Avis :**

- `avis_id` : Identifiant unique (Clé primaire)
- `note` : Évaluation numérique laissée au conducteur.
- `commentaire` : Message textuel rédigé.
- `statut` : État de modération de l'avis par l'administrateur.

- **Role :**

- `role_id` : Identifiant unique du rôle.
- `libelle` : Type d'accès (administrateur, voyageur).

- **Marque :**

- `marque_id` : Identifiant unique.
- `libelle` : Nom du constructeur automobile (ex: Renault, Peugeot).

- **Configuration & Parametre :**

- `id_configuration`, `parametre_id`, `propriete`, `valeur` : Données techniques destinées aux réglages globaux de l'application.

3.3 Règles de Gestion et Cardinalités (MCD)

Les cardinalités traduisent textuellement les liaisons graphiques de l'annexe 1 :

1. Relation [Utilisateur] --(possede)-- [role] :

- Un Utilisateur possède obligatoirement au moins un rôle (1,N).
- Un Rôle peut être attribué à aucun ou plusieurs utilisateurs (0,N).

2. Relation [Utilisateur] --(gere)-- [Voiture] :

- Un Utilisateur peut gérer aucun ou plusieurs véhicules (0,N) s'il fait uniquement du covoiturage en tant que passager.
- Une Voiture est obligatoirement gérée par un et un seul utilisateur (1,1).

3. Relation [Utilisateur] --(participe)-- [Covoiturage] :

- Un Utilisateur peut participer à aucun ou plusieurs covoiturages (0,N), que ce soit en tant que conducteur ou passager.
- Un Covoiturage enregistre la participation de plusieurs utilisateurs (0,N).

4. Relation [Voiture] --(utilise)-- [Covoiturage] :

- Une Voiture peut être utilisée pour aucun ou plusieurs covoiturages (0,N).
- Un Covoiturage utilise obligatoirement une et une seule voiture (1,1).

5. Relation [Voiture] --(detient)-- [Marque] :

- Une Voiture détient obligatoirement une seule marque (1,1).
- Une Marque peut être détenue par plusieurs voitures (0,N).

6. Relation [Utilisateur] --(depose)-- [avis] :

- Un Utilisateur peut déposer aucun ou plusieurs avis (0,N) après ses voyages.
- Un Avis est obligatoirement déposé par un unique utilisateur (1,1).

7. Gestion Technique [configuration] --(dispose)-- [parametre] :

- Une configuration dispose de plusieurs paramètres globaux (0,N), et chaque paramètre est lié de manière stricte à une configuration (1,1).

CHAPITRE 4 : DIAGRAMME D'UTILISATION (USE CASE)

Ce chapitre cartographie les fonctionnalités de l'application EcoRide en définissant les droits et les interactions de chaque acteur (type d'utilisateur) avec le système.

4.1 Définition des Acteurs

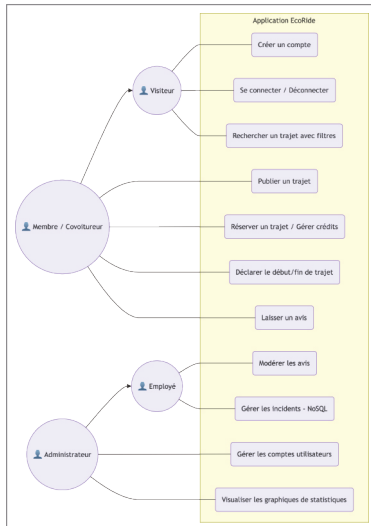


Figure 2 : Diagramme d'Utilisation (Use Case)

- **Visiteur (Non connecté) :** Tout internaute arrivant sur la plateforme EcoRide.
- **Utilisateur Connecté :** Acteur général ayant un compte validé.
Il se décline en deux sous-rôles :
 - **Passager :** Voyageur qui recherche et réserve des trajets.
 - **Conducteur :** Automobiliste qui propose des trajets et enregistre son véhicule.
- **Administrateur :** Responsable de la modération, de la gestion des comptes et du suivi global de la plateforme.

4.2 Description Textuelle des Cas d'Utilisation

A. Le Visiteur

Le visiteur possède des droits limités visant à l'inciter à rejoindre la plateforme :

- **Consulter et rechercher des trajets :** Saisir une ville de départ, une ville d'arrivée et une date pour voir les offres disponibles.
- **Créer un compte (Inscription) :** Renseigner ses informations personnelles pour devenir un utilisateur du service.

B. L'Utilisateur Connecté (Général)

Une fois authentifié, l'utilisateur a accès aux fonctionnalités communes :

- **Se connecter / Se déconnecter :** Accéder de manière sécurisée à son espace personnel.

- **Gérer son profil** : Modifier ses informations, son numéro de téléphone ou son mot de passe.

C. Le Passager

- **Réserver un trajet** : Bloquer une ou plusieurs places sur un trajet disponible.
- **Annuler une réservation** : Demander l'annulation d'un trajet programmé (ce qui remet les places en disponibilité).
- **Laisser un avis** : Évaluer et commenter la prestation d'un conducteur une fois le trajet marqué comme "Terminé".

D. Le Conducteur

- **Enregistrer un véhicule** : Ajouter la marque, le modèle et l'immatriculation de sa voiture (requis pour publier un trajet).
- **Proposer un trajet** : Publier un nouvel itinéraire avec date, heure, prix et places disponibles.
- **Annuler un trajet** : Retirer un trajet de la vente (ce qui bascule automatiquement les réservations associées en statut "Annulé par le conducteur").
- **Déclarer la fin du trajet** : Valider manuellement que le voyage est terminé pour clore la session et débloquer les avis.

E. L'Administrateur

L'administrateur possède un contrôle total sur la conformité de l'application :

- **Modérer les avis** : Valider ou rejeter les commentaires laissés par les passagers avant leur publication officielle.
- **Gérer les utilisateurs** : Activer, suspendre ou supprimer des comptes en cas de comportement inapproprié.
- **Visualiser les statistiques** : Consulter les volumes de trajets et de réservations de la plateforme.

CHAPITRE 5 : DIAGRAMME DE SÉQUENCE TECHNIQUE

Le diagramme de séquence permet de visualiser de manière chronologique les interactions entre les différents composants de l'application (l'utilisateur, l'interface graphique et le serveur) lors d'un cas d'utilisation précis.

5.1 Scénario choisi : Réservation d'un trajet par un Passager

Ce scénario décrit les étapes techniques qui se déroulent lorsque le passager clique sur "Réserver" jusqu'à la confirmation de sa réservation et la mise à jour des places dans la base de données.

5.2 Description Textuelle des Échanges (Chronologie)

- 1. Action Utilisateur :** Le Passager clique sur le bouton "Réserver une place" depuis la page des détails d'un trajet sur l'interface (Front-end).
- 2. Requête HTTP :** L'interface envoie une requête POST sécurisée au script PHP de gestion des réservations (Back-end), contenant l'identifiant du trajet et l'identifiant du passager issu de la session.
- 3. Vérification de Session :** Le serveur PHP vérifie via la superglobale `$_SESSION` que l'utilisateur est bien connecté et possède le rôle de passager.
- 4. Contrôle des Disponibilités (Base de données MySQL) :**
 - Le serveur exécute une requête SQL `SELECT` (via PDO) pour vérifier s'il reste des places disponibles dans la table `Covoiturage`.
 - La base de données renvoie le nombre de places restantes.
- 5. Traitement Métier (PHP) :**
 - *Si le nombre de places est suffisant :* Le serveur prépare la création de la réservation.
 - *Si le nombre de places est insuffisant :* Le processus s'arrête et renvoie une erreur à l'interface.
- 6. Enregistrement de la Réservation (Base de données MySQL) :**
 - Le serveur exécute une transaction SQL pour insérer une nouvelle ligne dans la table `participe` (ou table des réservations) avec le statut "Validée".
 - Simultanément, le serveur exécute une requête `UPDATE` pour décrémenter le champ `nb_place` de la table `Covoiturage`.
- 7. Confirmation de la Base de données :** MySQL confirme au serveur PHP que les requêtes ont été exécutées avec succès (Commit de la transaction).
- 8. Réponse au Client :** Le serveur PHP génère une réponse (HTML ou JSON) confirmant le succès de l'opération.
- 9. Affichage Final :** L'interface graphique (Front-end) se met à jour et affiche un message de succès à l'écran du passager ("Votre réservation a bien été prise en compte »).

5.3 Scénario alternatif : Consultation et gestion des incidents par un Employé (MongoDB - US 12)

Ce second scénario illustre l'architecture hybride de l'application EcoRide. Il décrit les interactions techniques lors de l'accès à l'espace de modération des litiges, mettant en évidence l'utilisation du stockage non-relationnel pour la traçabilité des incidents.

5.4 Description Textuelle des Échanges (Chronologie du traitement NoSQL)

- 1. Action Utilisateur :** L'Employé clique sur le bouton "Gérer les incidents" depuis son tableau de bord (Front-end).
- 2. Requête HTTP :** Le navigateur envoie une requête HTTP GET sécurisée vers le script `/incidents.php` (Back-end), en transmettant le cookie de session de l'utilisateur.
- 3. Vérification de Sécurité (PHP) :** Le serveur PHP intercepte la demande et analyse la superglobale `$_SESSION[ 'user_role' ]` pour valider les droits d'accès.
 - *Cas d'échec :* Si le rôle requis n'est pas trouvé, une redirection immédiate ou une erreur 403 (Accès refusé) est renvoyée au navigateur.
 - *Cas de succès :* Si l'utilisateur est bien authentifié comme Employé, le script poursuit son exécution.
- 4. Appel à la base NoSQL (MongoDB) :** Le code PHP instancie la classe native `MongoDB\Driver\Manager`. Il prépare et exécute une requête de lecture (Query) sur la collection dédiée aux incidents.
- 5. Réponse du SGBD NoSQL :** Le serveur MongoDB traite la demande à haute performance et retourne un curseur de données au format BSON contenant la liste des litiges enregistrés.
- 6. Traitement et Rendu (PHP) :** Le serveur PHP réceptionne le flux BSON, convertit les documents en objets manipulables, puis exécute une boucle d'affichage pour injecter dynamiquement les données dans le template HTML.
- 7. Affichage Final :** Le serveur PHP renvoie la page web complète au format HTML. Le navigateur l'affiche à l'écran de l'employé sous la forme d'un tableau de bord de modération propre et lisible.

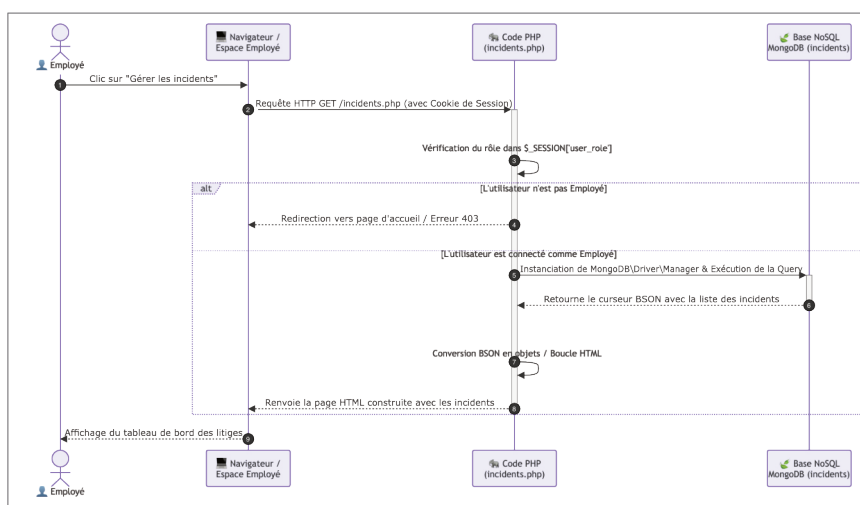


Figure 3 : Diagramme de séquence

CHAPITRE 6 : DOCUMENTATION DU DÉPLOIEMENT DE L'APPLICATION

Ce chapitre détaille la procédure technique permettant de migrer l'application EcoRide de l'environnement de développement local (MAMP) vers un serveur de production accessible sur Internet.

6.1 Choix de l'Hébergement et Prérequis

Pour accueillir l'architecture hybride d'EcoRide (MySQL pour la gestion relationnelle et MongoDB pour la gestion des logs), le choix s'est porté sur un hébergement Cloud ou un Serveur Privé Virtuel (VPS) permettant d'installer et de configurer ces deux moteurs de base de données.

Les prérequis sur le serveur de production sont :

- Un serveur web (Apache ou Nginx).
- PHP (version identique à celle utilisée sur MAMP) avec l'extension **PDO MySQL** et le driver **MongoDB** activés.
- Un accès SSH ou FTP pour le transfert des fichiers.

6.2 Étapes de Déploiement

Étape 1 : Exportation et Migration des Bases de Données

1. **MySQL** : Depuis l'interface phpMyAdmin locale, effectuer un EXPORT de la base de données EcoRide au format `.sql`. Sur le serveur de production, créer une nouvelle base de données et y importer ce fichier.
2. **MongoDB** : Utiliser la commande `mongodump` en local pour exporter la collection des logs de l'application, puis la restaurer sur le serveur distant à l'aide de `mongorestore`.

Étape 2 : Transfert des Fichiers de l'Application

1. Nettoyer le dossier de développement local en supprimant les fichiers temporaires.
2. À l'aide d'un client FTP (comme FileZilla) ou d'une commande de synchronisation sécurisée (Git / SCP), transférer l'intégralité du code source (fichiers PHP, scripts, feuilles de style CSS, images, etc.) vers le répertoire public du serveur distant (souvent nommé `public_html` ou `var/www/html`).

Étape 3 : Ajustement des Variables de Configuration (Environnement)

Une fois les fichiers en ligne, il est crucial de modifier le fichier de configuration de l'application (souvent un fichier `.env` ou un script de connexion globale) :

- Mettre à jour les identifiants de connexion MySQL (hôte distant, nom de la base, utilisateur et mot de passe de production).

- Mettre à jour la chaîne de connexion (URI) pour MongoDB.
- Désactiver l'affichage des erreurs PHP à l'écran (`display_errors = Off` dans le fichier `php.ini` de production) pour des raisons de sécurité, et rediriger ces erreurs vers un fichier de log privé.

Étape 4 : Tests de Validation (Recette)

Réaliser une série de tests fonctionnels directement sur l'URL publique pour s'assurer du bon fonctionnement du site :

- Création d'un compte utilisateur (Inscription).
- Recherche et réservation d'un trajet de covoiturage.
- Vérification dans la base de données de production que les écritures s'effectuent correctement et de manière sécurisée.